EECE 7205: Fundamentals of Computer Engineering
Airport BST Mini Project

Nashita Hasan, Katherine Tse
Email: hasan.nas@northeastern.edu, tse.kat@northeastern.edu
NUID: 002750189, 003936627
Due: October 29, 2026

1 Task 1 Load the data and build the tree (insert)

Algorithm 1.1 Load data and build the tree (insert)

```
1: class Airport
2:     Public:
3:         Name
4:         City
5:         Country
6:         IATA Code
7:         ICAO Code
8:         Latitude
9:         Longitude
10:        Left
11:        Right
12:    function AirportConstructor(name, city, country, IATA, ICAO, lat, long)
13:        Name ← name
14:        City ← city
15:        Country ← country
16:        IATACode ← IATA
17:        ICAOCode ← ICAO
18:        Latitude ← lat
19:        Longitude ← long
20:        Left ← NULL
21:        Right ← NULL
22:    end function
23: end class
24: class AirportBST
25:    Private:
26:        root
27:    function SumDepth(node, d, totalNodes(pass by reference))
28:        if node = NULL then
29:            return 0
30:        end if
31:        totalNodes ← totalNodes + 1
32:        return d + SumDepth(node.left, d + 1, totalNodes)+
33:            SumDepth(node.right, d + 1, totalNodes)
34:    end function
```

Algorithm 1.1 Load data and build the tree (insert) [continued]

```
35:     function HEIGHT(node)
36:         if node = NULL then
37:             return -1
38:         end if
39:         leftHeight ← HEIGHT(node.left)
40:         rightHeight ← HEIGHT(node.right)
41:         if leftHeight < rightHeight then
42:             return 1 + rightHeight
43:         else
44:             return 1 + leftHeight
45:         end if
46:     end function
47:     Public:
48:     function AIRPORTBSTCONSTRUCTOR
49:         root ← NULL
50:     end function
51:     function INSERT(newAirport)
52:         if root = NULL then
53:             root ← newAirport
54:             return Added
55:         end if
56:         curr ← root
57:         while TRUE do
58:             if key < curr.key then
59:                 if curr.Left = NULL then
60:                     curr.Left ← newNode
61:                     return Added
62:                 end if
63:                 curr ← curr.Left
64:             else if key > curr.key then
65:                 if curr.Right = NULL then then
66:                     curr.Right ← newNode
67:                     return Added
68:                 end if
69:                 curr ← curr.Right
70:             else
71:                 return Duplicate
72:             end if
73:         end while
74:     end function
75:     function GETHEIGHT
76:         return HEIGHT(root)
77:     end function
```

Algorithm 1.1 Load data and build the tree (insert) [continued]

```
78:     function avgDepth
79:         totalNodes ← 0
80:         sumDepth ← sumDepth(root, 0, totalNodes)
81:         return sumDepth ÷ totalNodes
82:     end function
83: end class
84: function LoadData(airportFile)
85:     Airports[ ]
86:     while airportFile.ParseNextLine ≠ NULL do
87:         line ← airportFile.ParseNextLine
88:         fields[ ]
89:         while line.ParseNextFeild ≠ NULL do
90:             fields.APPEND(line.ParseNextFeild)
91:         end while
92:         IATA ← fields[4]
93:         if IATACode ≠ \N then
94:             name ← fields[1]
95:             city ← fields[2]
96:             country ← fields[3]
97:             ICAO ← fields[5]O
98:             lat ← fields[6]
99:             long ← fields[7]
100:             Airports.APPEND(
101:                 AirportConstructor(name, city, country, IATA, ICAO, lat, long))
102:         end if
103:     end while
104:     return Airports
105: end function
106: function BuildTree(Airports)
107:     n ← Airports.SIZE
108:     Added ← 0
109:     Duplicates ← 0
110:     AirportTree ← AirportBSTConstructor( )
111:     for i ← 0 to n − 1 do
112:         if AirportTree.Insert(Airports[i]) = Added then
113:             Added ← Added + 1
114:         else
115:             Duplicates ← Duplicates + 1
116:         end if
117:     end for
118:     Height ← AirportTree.getHeight( )
119:     Depth ← AirportTree.avgDepth( )
120:     PRINT( Keys Inserted: {added}, Duplicates: {duplicates},
121:         Tree Height: {height}, Average Depth: {Depth})
122:     return AirportTree
123: end function
```

2 Task 2 Search

Algorithm 2.1 Search (inside the AirportBST Class

```
1: Public:
2: function FIND(code)
3:     curr ← root
4:     comparisons ← 0
5:     PRINT("Path : ")
6:     while curr ≠ NULL do
7:         comparisons ← comparisons + 1
8:         PRINT(curr.IATACode)
9:         if code = curr.IATACode then
10:            PRINT("(Comparisons : ", comparisons, ")", {new line})
11:            PRINT(curr.IATACode, curr.Name, curr.City, curr.Country,
12:              curr.ICAOCode, curr.Latitude, curr.Longitude)
13:            return curr
14:        else if code < curr.IATACode then
15:            curr ← curr.left
16:        else
17:            curr ← curr.right
18:        end if
19:        Print(" -> ")
20:    end while
21:    PRINT("(NULL) not found")
22: end function
23: function FINDMIN
24:    curr ← root
25:    while  docurr.left ≠ NULL curr ← curr.left
26:    end while
27:    PRINT("Minimum : ", curr.IATACode)
28: end function
29: function FINDMAX
30:    curr ← root
31:    while  docurr.right ≠ NULL
32:        curr ← curr.right
33:    end while
34:    PRINT("Maximum : ", curr.IATACode)
35: end function
```

3 Task 3 Delete

Algorithm 3.1 Delete (inside the AirportBST Class

```
1: Private:
2: function recursiveRemove(node, code)
3:     if node = NULL then
4:         return NULL
5:     end if
6:     if code < node.IATACode then
7:         node.left ← recursiveRemove(node.left, code)
8:     else if code > node.IATACode then
9:         node.right ← recursiveRemove(node.right, code)
10:    else
11:        if node.left = NULL then
12:            return node.right
13:        end if
14:        if node.right = NULL then
15:            return node.left
16:        end if
17:        s ← node.right
18:        while s.left ≠ NULL do
19:            s ← s.left
20:        end while
21:        Node.Name ← s.Name
22:        Node.City ← s.City
23:        Node.Country ← s.Country
24:        Node.IATACode ← s.IATACode
25:        Node.ICAOCode ← s.ICAOCode
26:        Node.Latitude ← s.Latitude
27:        Node.Longitude ← s.Longitude
28:        node.right ← recursiveRemove(node.right, s.IATACode)
29:    end if
30:    return node
31: end function
32: Public:
33: function remove(code)
34:    recursiveRemove(root, code)
35: end function
```

Algorithm 3.1 Delete [continued]

```
36: function printLevelOrder(levels)
37:     if root = NULL then
38:         PRINT(Empty Tree)
39:         return
40:     end if
41:     queue ← root
42:     currLevel ← 1
43:     while queue NOT empty AND currLevel ≤ levels do
44:         levelSize = queue.SIZE()
45:         for i ← 0 to levelSize − 1 do
46:             currNode ← queue.POP
47:             PRINT(currNode.IATACode, " ")
48:             if currNode.left ≠ NULL then
49:                 queue.ADD(currNode.left)
50:             end if
51:             if currNode.right ≠ NULL then
52:                 queue.ADD(currNode.right)
53:             end if
54:         end for
55:         PRINT({newline})
56:         currLevel ← currLevel + 1
57:     end while
58: end function
```

4 Task 4 Traversals and validation

Algorithm 4.1 Traversal and Validation (inside the AirportBST Class

```
1: Private:
2: function recursiveList(node)
3:     if node = NULL then
4:         return
5:     end if
6:     recursiveList(node.left)
7:     PRINT(node.IATACode, node.Name, node.City, node.Country,
8:       node.ICAOCode, node.Latitude, node.Longitude, {new line})
9:     recursiveList(node.right)
10: end function
11: function recursiveSave(node, File)
12:     if node = NULL then
13:         return
14:     end if
15:     File.WRITEnode.IATACode
16:     recursiveSave(node.left, File)
17:     recursiveSave(node.right, File)
18: end function
19: function RecursiveValidate(node, minKey, maxKey)
20:     if node = NULL then
21:         return TRUE
22:     end if
23:     if minKey ≠ NULL and node.IATACode ≤ minKey then
24:         return FALSE
25:     end if
26:     if maxKey ≠ NULL and node.IATACode ≥ maxKey then
27:         return FALSE
28:     end if
29:     leftValid ← RecursiveValidate(node.left, minKey, node.IATACode)
30:     rightValid ← RecursiveValidate(node.right, node.IATACode, maxKey)
31:     return leftValid AND rightValid
32: end function
```

Algorithm 4.1 Traversal and Validation [continued]

```
33: Public:
34: function LIST
35:     RECURSIVELIST(root)
36: end function
37: function SAVE(File)
38:     if File = INVALID then
39:         PRINT(Invalid File)
40:         return
41:     end if
42:     RECURSIVESAVE(root, File)
43: end function
44: function LOAD(File)
45:     if File = INVALID then
46:         PRINT(Invalid File)
47:         return
48:     end if
49:     newBST ← AIRPORTBSTCONSTRUCTOR( )
50:     while File.ParseNextLine ≠ NULL do
51:         code ← File.ParseNextLine
52:         node ← FIND(code)
53:         newBST.INSERT(node)
54:     end while
55:     return newBST
56: end function
57: function VALIDATE
58:     return RECURSIVEVALIDATE(root, NULL, NULL)
59: end function
```

5 Task 5 Range Queries

Algorithm 5.1 Range Queries

```
1: Private:
2: function recursiveRange(node, From, To, nodesVisited(passed by reference))
3:     if node = NULL then
4:         return
5:     end if
6:     nodesVisited ← nodesVisited + 1
7:     if node.left.IATACode ≥ From then
8:         recursiveRange(node.left, From, To, nodesVisited)
9:     end if
10:    PRINT(node.IATACode)
11:    if node.right.IATACode ≤ To then
12:        recursiveRange(node.right, From, To, nodesVisited)
13:    end if
14: end function
15: Public:
16: function range(From, To)
17:    nodesVisited ← 0
18:    recursiveRange(Root, From, To, nodesVisited)
19:    return nodesVisited
20: end function
```

6 Task 6 The Shape Experiment

Algorithm 6.1 Shape Experiment

```
1: Private:
2: function recursiveBuildBalanced(airports, low, high)
3:     if low > high then
4:         return NULL
5:     end if
6:     mid ← (low + high)/2
7:     node ← airports[mid]
8:     node.left ← recursiveBuildBalanced(airports, low, mid − 1)
9:     node.right ← recursiveBuildBalanced(airports, mid + 1, high)
10:    return node;
11: end function
12: Public:
13: function buildBalancedTree(airports, n)
14:    root ← recursiveBuildBalanced(airports, 0, n − 1)
15: end function
16: function main( )
17:    airports ← LoadData(airportFile)
18:    airportBST ← BuildTree(airports)
19:    start ← CLOCK()
20:    for i ← 10,000 to 1 do
21:        code ← generateRandomCode()
22:        find(code)
23:    end for
24:    end ← CLOCK()
25:    fileOrderTime ← end − start
26:    airports.SORT()
27:    sortedAirportBST ← BuildTree(airports)
28:    start ← CLOCK()
29:    for i ← 10,000 to 1 do
30:        code ← generateRandomCode()
31:        find(code)
32:    end for
33:    end ← CLOCK()
34:    sortedTime ← end − start
35:    balancedTree ← AirportBSTConstructor( )
36:    balancedTree.buildBalancedTree(airports, airports.SIZE())
```

Algorithm 6.1 Shape Experiement [continued]

```
37:     start ← CLOCK()
38:     for i ← 10,000 to 1 do
39:         code ← generateRandomCode()
40:         FIND(code)
41:     end for
42:     end ← CLOCK()
43:     balancedTime ← end − start
44: end function
```